Sparse Autoencoder Baselines for Audio Representation

Mathematical formulation, exact architecture, training protocol, and measured comparison of three models

LRAC research project

27 October 2026

Abstract

The research objective is to discover a representation that reconstructs a particular audio signal using as few retained scalar values as possible, without requiring a named transform or interpretable feature labels. This report specifies the first controlled experiment: a sparse nonlinear convolutional autoencoder, its dense-trained nonlinear control, and a sparse affine analysis–synthesis control. All three contain 1,002,369 trainable parameters and were trained for 20,000 updates on the same speaker-balanced ten-hour subset of LibriSpeech. The affine control currently gives the strongest reconstruction results on a common 200-utterance development probe. This is an achieved sparsity–distortion comparison within a restricted model family, not a certificate of the sparsest possible representation, a lossless transform, or a measured audio bitrate. The report connects every architecture and objective to the implementation, reports the observed results, and identifies the remaining research questions.

Experimental status. Three training runs and validation reports are complete. Results use five full utterances from each of 40 `dev-clean` speakers. These same utterances selected checkpoints. Final `test-clean` reconstruction scores have not been computed.

Contents

1 Research question and scope **3**
1.1 Why these three models? . . . 3

2 Shared representation and architecture **4**
2.1 Inputs, lengths, and dense code capacity . . . 4
2.2 Layer definitions . . . 4
2.3 Encoder: exact operation order . . . 4
2.4 Decoder: exact operation order . . . 5
2.5 Global hard support selection . . . 6

3 The three mathematical models **6**
3.1 S-NL: sparse nonlinear autoencoder . . . 6
3.2 D-NL: dense-trained nonlinear control . . . 6
3.3 S-AF: sparse affine analysis–synthesis control . . . 7

4 Training objective and optimization **7**
4.1 Reconstruction objective actually used . . . 7
4.2 Optimizer and update schedule . . . 8
4.3 Validation and model selection . . . 8

5 Dataset, partitioning, and leakage controls **8**
5.1 Source corpus and inspected local data . . . 8
5.2 Loading and sampling . . . 9
5.3 Audits and test discipline . . . 9

6 Measurements and mathematical definitions **9**
6.1 Sparsity and complete representation accounting . . . 9
6.2 Waveform reconstruction . . . 10
6.3 Spectral, transient, and high-band diagnostics . . . 10
6.4 Speech quality and intelligibility . . . 11
6.5 Aggregation and smallest tested support . . . 11

7 Measured development results **11**
7.1 Completion and checkpoint objective . . . 11
7.2 Support–distortion curves . . . 12
7.3 Tolerance attainment . . . 12
7.4 What the observations currently support . . . 14

8 Reproduction, files, and code correspondence **14**
8.1 Minimal workflow . . . 14
8.2 Implementation map . . . 15

9 Limits and next research decisions **16**
9.1 What the baseline does not establish . . . 16
9.2 Questions made sharper by the three runs . . . 16

1 Research question and scope

For an audio vector $x \in \mathbb{R}^N$, the desired system produces a short representation from which a decoder reconstructs x to a specified accuracy. It need not recognize a Fourier transform, label phonemes, or explain the meaning of a feature. A distortion-dependent ideal, after fixing an admissible decoder family $\mathcal{G}$, is

$$K^*_{\mathcal{G}}(x, \varepsilon) = \min_{g \in \mathcal{G},\, z} \|z\|_0 \quad \text{subject to} \quad d(x, g(z)) \le \varepsilon. \tag{1}$$

Here $\|z\|_0$ counts nonzero scalar coordinates, not time frames, channels, or semantic features. Different distortion measures can give different answers.

Equation (1) is meaningful only if the decoder, its admissible dependence on the sample, precision, and side information are specified. An unconstrained per-sample decoder can simply memorize x and reconstruct it from an empty code. Even an orthogonal transform chosen separately for $x \neq 0$ can map it to $(\|x\|_2, 0, \ldots, 0)$; the transform then contains its direction. These are elementary counterexamples, not proposed solutions. Description length offers a complementary accounting perspective [1].

For this baseline, one encoder and one decoder are learned from training recordings and then frozen. The code and its support change for each input; network weights do not. For a fixed decoder g_θ, the encoder is an amortized proposal for a sparse code, rather than a solver that proves the minimum:

$$z_k(x) = \mathcal{H}_k(f_\phi(x)), \qquad \widehat{x}_k = g_\theta(z_k(x)). \tag{2}$$

We measure the attainable pairs $(\|z_k(x)\|_0, d(x, \widehat{x}_k))$. If a code satisfies a tolerance, its support is an *upper bound* on the best support attainable with that decoder and the same numerical conventions. A failed encoder proposal gives no lower bound on other possible codes.

1.1 Why these three models?

The comparison changes two concrete choices while preserving the data, training budget, channel counts, parameter count, and evaluation code:

ID	Encoder/decoder activation	Training support	Question
S-NL	Leaky ReLU	Seven sampled fractions	Can reconstruction training organize values to survive severe pruning?
D-NL	Leaky ReLU	All values	What changes if sparsity is imposed only after dense training?
S-AF	Identity	Seven sampled fractions	Does the nonlinear network help relative to an affine analysis–synthesis map?

The sparse autoencoder follows the hard activation-selection idea of k-sparse autoencoders [2], adapted here to signed convolutional audio codes, magnitude ranking, and multiple whole-recording budgets. It is not an exact reproduction of that paper's network or experiments. The residual connections follow the general residual-learning construction [3]; the specific one-dimensional architecture below is this project's baseline design.

2 Shared representation and architecture

2.1 Inputs, lengths, and dense code capacity

Inputs are decoded mono waveforms at $F_s = 16{,}000$ Hz, represented as float32 tensors of shape $(B, 1, N_{\max})$, together with valid lengths N_i. There is no per-recording gain normalization, transcript input, speaker embedding, or target-dependent information supplied to the decoder.

Each valid waveform is right-padded to

$$\overline{N}_i = 256 \left\lceil \frac{N_i}{256} \right\rceil, \quad T_i = \left\lceil \frac{N_i}{256} \right\rceil, \quad C = 256, \quad M_i = CT_i. \tag{3}$$

The encoder returns $U_i \in \mathbb{R}^{T_i \times 256}$. Flattening gives $u_i \in \mathbb{R}^{M_i}$. For a two-second training crop, $N = 32{,}000$, $T = 125$, and $M = 32{,}000$: the *dense* code has the same number of scalar values as the waveform. Sparsity, rather than a smaller dense tensor, supplies the bottleneck. For other lengths, M can be slightly larger than N because of the last partially occupied time step.

Batch padding is excluded. The implementation encodes equal-valid-length items together so that a longer item in the same batch cannot alter a shorter item's boundary activations. The coefficient mask is

$$V_i[t, c] = \mathbf{1}\{256t < N_i\}, \qquad c = 0, \ldots, 255. \tag{4}$$

All 256 channels in a partially occupied final step count toward valid capacity. They are not silently discarded from sparsity accounting.

2.2 Layer definitions

Write $\mathcal{C}^{a\to b}_{q,s,p}$ for a biased 1D convolution with kernel length q, stride s, and padding p. Its cross-correlation convention is

$$[\mathcal{C}(h)]_{b,t} = b_b + \sum_{a=1}^{C_{\text{in}}} \sum_{j=0}^{q-1} W_{b,a,j}\, h_{a,st+j-p}, \tag{5}$$

with zero values outside the valid padded extent. Let $\mathcal{T}^{a\to b}_{9,4,4,3}$ denote the corresponding transposed-convolution layer with output padding 3. For input length L, its output length is

$$(L-1)4 - 2(4) + (9-1) + 3 + 1 = 4L. \tag{6}$$

Transposed convolution is a learned synthesis operation; it is not assumed to invert the encoder convolution.

For S-NL and D-NL,

$$\sigma(v) = \begin{cases} v, & v \geq 0, \\ 0.2v, & v < 0. \end{cases} \qquad \mathcal{R}_c(h) = h + \mathcal{C}^{c\to c}_{3,1,1}\Big(\sigma\Big(\mathcal{C}^{c\to c}_{3,1,1}(h)\Big)\Big). \tag{7}$$

The two convolutions inside $\mathcal{R}_c$ have separate parameters. There is no additional activation after the residual sum. In S-AF, every occurrence of σ is replaced by the identity, including the activation inside each residual block.

2.3 Encoder: exact operation order

For channel sequence $(1, 32, 64, 128, 128)$, stage j is

$$h_j = \mathcal{R}_{c_j}\Big(\sigma\Big(\mathcal{C}^{c_{j-1}\to c_j}_{9,4,4}(h_{j-1})\Big)\Big), \qquad h_0 = x_{\text{padded}}, \quad j = 1, \ldots, 4. \tag{8}$$



Figure 1: The three models share this path. No waveform bypass is available to the decoder. Support positions, the sparse values, the shared model, and length metadata are required for reconstruction.

Stage	Operations	Output $C \times L$ for 2 s	Parameters
E1	Conv $1 \to 32$; σ; $\mathcal{R}_{32}$	32×8000	6,528
E2	Conv $32 \to 64$; σ; $\mathcal{R}_{64}$	64×2000	43,200
E3	Conv $64 \to 128$; σ; $\mathcal{R}_{128}$	128×500	172,416
E4	Conv $128 \to 128$; σ; $\mathcal{R}_{128}$	128×125	246,144
Latent	Conv $128 \to 256$, kernel 1	256×125	33,024
Encoder total			501,312

Table 1: Encoder geometry and trainable counts, including biases. All non-1×1 stage convolutions use kernel 9, stride 4, padding 4. Residual convolutions use kernel 3, stride 1, padding 1.

A final biased 1×1 convolution maps 128 channels to 256. The result is transposed from (B, C, T) to (B, T, C) for support selection. There is no latent positivity constraint.

The temporal hop is $4^4 = 256$ samples, or 16 ms. The maximum receptive field of a dense encoder coordinate is 2,041 samples, approximately 127.6 ms. To verify this, start with receptive field $r = 1$ and jump $j = 1$. At each stage add $8j$ for the stride-4 convolution, set $j \leftarrow 4j$, then add $4j$ for the two residual convolutions. Thus candidate values are locally computed, although support selection compares candidates across the whole recording.

2.4 Decoder: exact operation order

The decoder first applies a biased 1×1 convolution $256 \to 128$. For source–target channel pairs $(128, 128)$, $(128, 64)$, $(64, 32)$, and $(32, 1)$, it then applies

$$h \longmapsto \mathcal{T}^{a\to b}_{9,4,4,3}(\sigma(\mathcal{R}_a(h))), \tag{9}$$

followed by σ if $b \neq 1$. There is no final activation or clipping. Decoding takes only the T_i valid latent steps and crops the output back to N_i.

Stage	Operations	Output $C \times L$ for 2 s	Parameters
Projection	Conv $256 \to 128$, kernel 1	128×125	32,896
D1	$\mathcal{R}_{128}$; σ; TConv $128 \to 128$; σ	128×500	246,144
D2	$\mathcal{R}_{128}$; σ; TConv $128 \to 64$; σ	64×2000	172,352
D3	$\mathcal{R}_{64}$; σ; TConv $64 \to 32$; σ	32×8000	43,168
D4	$\mathcal{R}_{32}$; σ; TConv $32 \to 1$	1×32000	6,497
Decoder total			501,057
Complete model			1,002,369

Table 2: Decoder geometry. Each transposed convolution uses kernel 9, stride 4, padding 4, output padding 3. There is no encoder–decoder skip connection, normalization layer, dropout, or tied analysis/synthesis weight.

2.5 Global hard support selection

For a requested fraction $\rho \in [0, 1]$, the budget for example i is

$$k_i = \lceil \rho M_i \rceil. \tag{10}$$

Alternatively a nonnegative integer budget K gives $k_i = \min(K, M_i)$. Let $S_k(u)$ be the indices of the k largest $|u_j|$ among valid coordinates. Then

$$[\mathcal{H}_k(u)]_j = u_j \mathbf{1}\{j \in S_k(u)\}. \tag{11}$$

The mask is chosen separately for each waveform, across every valid time step and channel. It is neither a per-frame quota nor a fixed selection of channel types. Signs and amplitudes are retained. $\rho = 0$ retains no values; $\rho = 1$ retains all valid values. Selected positions that already equal zero do not increase actual ℓ_0, so actual support can be smaller than the selected budget. Equal-magnitude ties follow the numerical library's top-k behavior.

Magnitude selection minimizes $\|u - v\|_2^2$ over k-sparse v in *latent coordinates*. It does not generally minimize waveform error after a learned decoder. Even for an affine decoder $Av + a$, the relevant error involves $A^\top A$ and possible residual error. Only special geometries such as orthonormal synthesis with the appropriate analysis map make coefficient magnitude an exact waveform support rule. Neither orthogonality nor invertibility is imposed here.

Away from selection boundaries, automatic differentiation treats the mask as fixed:

$$\frac{\partial [\mathcal{H}_k(u)]_j}{\partial u_\ell} = \mathbf{1}\{j \in S_k(u)\}\mathbf{1}\{j = \ell\}. \tag{12}$$

Thus selected amplitudes receive reconstruction gradients. The discrete change in ranking is not differentiated. This is ordinary hard selection, not a straight-through estimator for a separate learned gate. Convolutional weights are shared, and dense-budget batches also train coordinates that low-budget batches suppress.

3 The three mathematical models

3.1 S-NL: sparse nonlinear autoencoder

With nonlinear convolutional f_ϕ and g_θ,

$$\widehat{x}_\rho = g_\theta\Big(\mathcal{H}_{\lceil \rho M \rceil}(f_\phi(x))\Big), \qquad \rho \sim \text{Uniform}(\mathcal{P}), \quad \mathcal{P} = \{.01, .02, .05, .10, .25, .50, 1\}. \tag{13}$$

The model learns the candidate values and how to synthesize audio from them. The top-k rule is fixed by the experiment. No feature labels, predefined dictionaries, or independence requirements are used. A single fraction is sampled for each training microbatch; each example receives its own mask. The encoder does not receive ρ as an extra input.

The input changes the amplitudes and support, but there is no per-recording weight adaptation. Consequently this model tests whether a *shared* nonlinear representation can produce useful input-specific sparse codes. It does not search all transforms that might be optimal for a particular recording.

3.2 D-NL: dense-trained nonlinear control

D-NL uses the exact same architecture but trains only with $\rho = 1$:

$$\widehat{x}_{\text{train}} = g_\theta(f_\phi(x)). \tag{14}$$

At validation and evaluation it is subjected to the same hard selection in Eq. (11) and the same seven nonzero validation fractions as the sparse models. This holds the *checkpoint-selection criterion* fixed, but means D-NL is not selected specifically for its best dense reconstruction. A future dense-selected control would answer a different question and must be labeled separately.

S-NL versus D-NL isolates the effect of training through pruning, within this architecture and update budget. A low-budget advantage would not imply that S-NL is always better: dense-trained features may reconstruct better when most values are retained.

3.3 S-AF: sparse affine analysis–synthesis control

Replacing every Leaky ReLU by identity makes the dense encoder and decoder affine. For a fixed signal length and boundary convention, they can be expressed as

$$f_\phi(x) = Bx + b, \qquad g_\theta(z) = Az + a, \qquad \widehat{x}_\rho = A\mathcal{H}_{\lceil \rho M \rceil}(Bx + b) + a. \tag{15}$$

A and B here are *effective* convolutional operators, not explicitly stored dense matrices. Network biases remain trainable, so "affine" is the correct name. The complete system is nonlinear because its chosen support depends on x. Its training budgets and loss are those of S-NL.

The deep factorization and residual connections remain; removing activations does not turn training into a convex problem. A is not constrained to equal $B^\top$, to have orthogonal columns, or to be the inverse of B. There is no perfect-reconstruction filter-bank guarantee. This distinguishes the control from constrained sparsifying-transform and filter-bank learning [4, 5].

For fixed length, a retained coordinate corresponds to a column $A_{:,j}$ of the effective synthesis operator. This gives a conventional atom interpretation for S-AF. In S-NL, decoder interactions generally prevent a code-independent additive atom decomposition; a local derivative $\partial g_\theta / \partial z_j$ can describe sensitivity at one code, but not an invariant semantic feature.

4 Training objective and optimization

4.1 Reconstruction objective actually used

The common training power estimate is

$$P_{\mathrm{tr}} = \max\left(\frac{\sum_{x \in \mathcal{A}} \|x\|_2^2}{\sum_{x \in \mathcal{A}} N_x}, 10^{-8} \right) = 0.003196885499251754, \tag{16}$$

where $\mathcal{A}$ contains 64 deterministic, uncropped training examples selected using the bounded speaker-coverage loader. It uses training data only. For a batch with lengths N_i, the optimized loss is

$$\mathcal{L}_{\mathrm{rec}} = \frac{1}{P_{\mathrm{tr}}} \frac{\sum_{i=1}^{B} \sum_{n=0}^{N_i - 1} (x_i[n] - \widehat{x}_i[n])^2}{\sum_{i=1}^{B} N_i}. \tag{17}$$

This is masked waveform MSE divided by a *fixed* training power, not the average of each recording's energy-normalized NMSE. Louder recordings still contribute more absolute error. S-NL and S-AF minimize the expectation of Eq. (17) over training sampling, crops, and $\rho \in \mathcal{P}$. D-NL uses $\rho = 1$ in that expectation.

For these runs, spectral, ℓ_1, group-sparsity, and functional-interaction loss weights are all zero. Reported latent ℓ_1 and group norms are diagnostics, not optimized penalties. Sparsity is enforced by the support constraint, not by a shrinkage coefficient. A hard budget makes the number of retained values explicit, but it does not learn an optimal per-sample budget or a minimum-support proof.

4.2 Optimizer and update schedule

All three use AdamW [6]: learning rate $\eta = 2 \times 10^{-4}$, decoupled weight decay $\lambda = 10^{-4}$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, and numerical $\epsilon = 10^{-8}$. With w denoting all trainable parameters and q_t the globally clipped reconstruction gradient,

$$q_t = \nabla_w \mathcal{L}_t \cdot \min\left(1, \frac{1}{\|\nabla_w \mathcal{L}_t\|_2}\right), \tag{18}$$

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) q_t, \qquad v_t = \beta_2 v_{t-1} + (1-\beta_2) q_t^{\odot 2}, \tag{19}$$

$$w_t = (1-\eta\lambda) w_{t-1} - \eta \frac{m_t/(1-\beta_1^t)}{\sqrt{v_t/(1-\beta_2^t)} + \epsilon}. \tag{20}$$

The implementation uses PyTorch's gradient clipping with its numerical stabilizer. Weight decay applies to the full parameter list, including biases. There is no learning-rate schedule, mixed-precision training, or curriculum in these configurations.

Setting	Common value
Seed / trainable parameters	0 / 1,002,369
Training subset / waveform rate	Ten hours, 921 speakers / mono 16 kHz
Crop / batch / accumulation	At most 2 s / 8 / 1
Updates / data-loader workers	20,000 / 0
Validation / checkpoint interval	Every 500 updates; initialization also validated
Log interval	Every 20 updates, plus first update
Selected checkpoint	Lowest mean validation loss over seven nonzero budgets
Training precision	float32

The update budget processes 160,000 sampled examples per model, with replacement; this is not 160,000 unique recordings or a fixed number of corpus epochs. Identical seeding gives identical initial parameter tensors across the three model definitions. Changing activations does not change their parameter allocation.

4.3 Validation and model selection

The bounded validation loader takes recordings round-robin across speakers: 200 full utterances, five per each of 40 `dev-clean` speakers. Let x_i be these utterances. The checkpoint score is

$$\mathcal{V}(\phi, \theta) = \frac{1}{200\,|\mathcal{P}|} \sum_{i=1}^{200} \sum_{\rho \in \mathcal{P}} \frac{\left\| x_i - g_\theta(\mathcal{H}_{\lceil \rho M_i \rceil}(f_\phi(x_i))) \right\|_2^2}{N_i P_{\mathrm{tr}}}. \tag{21}$$

Each utterance and each budget have equal weight in this score. Training batches instead pool valid samples in Eq. (17). The implementation encodes each validation utterance once and decodes it at every budget. All three best checkpoints in the present comparison occur at update 20,000.

5 Dataset, partitioning, and leakage controls

5.1 Source corpus and inspected local data

LibriSpeech is a multi-speaker read-English audiobook corpus [7]. The inspected KaggleHub download contains the following official partition payloads:

Partition	Speakers	Source recordings	Hours	Experimental use
`train-clean-360`	921	104,014	~ 363.61	Source of training subset
`dev-clean`	40	2,703	5.388	Development only
`test-clean`	40	2,620	5.403	Held-out final test

The partition names are official labels; measured duration need not equal the rounded number in a label. There are zero speaker-ID intersections between these three partitions. LibriSpeech is therefore not a single-speaker corpus. Speaker separation helps test generalization to unseen narrators; it does not eliminate overfitting to read speech, recording conditions, or the development set.

The actual training manifest contains 3,329 source intervals from all 921 training speakers, totaling 35,999.99 seconds. Subset creation allocates approximately 36,000/921 seconds per speaker, uses deterministic per-speaker ordering under seed 0, and truncates the final selected interval at its assigned duration. A one-hour pilot subset exists but is not part of the main result tables.

5.2 Loading and sampling

Audio is decoded to float32 mono and resampled to the configured rate only when needed. No signal-derived normalization is fitted on held-out recordings. For an interval longer than 32,000 samples, training draws a uniformly positioned contiguous 32,000-sample crop. Shorter intervals retain their actual length and are padded only for collation.

Training samples source intervals with replacement using weight $1/n_s$ for every interval belonging to speaker s, where n_s is that speaker's interval count. Consequently speaker probability is balanced, followed by uniform interval selection within a speaker. This is not uniform sampling over every source sample: short and long intervals have the same weight within a speaker. The sampling and cropping policy is common to the three runs.

5.3 Audits and test discipline

Before training, the audit verifies partition roles, speaker identities, source extents, IDs, metadata, and cross-role duplicate decoded-audio hashes. All three runs passed. Source audio is not copied to the repository. Generated manifests and hash caches remain local.

The reported metrics use the same 200 development utterances that selected checkpoints. Their confidence intervals therefore describe variation over these development speakers, not an unbiased final generalization estimate after repeated model selection. Test recordings have been indexed and leakage-audited but have not been used for reconstruction evaluation, checkpoint selection, or fitting training statistics. The inspected cache lacks `test-other`; external speakers, other languages, music, and environmental audio remain untested.

6 Measurements and mathematical definitions

6.1 Sparsity and complete representation accounting

The primary achieved support and normalized counts are

$$K(x) = \sum_{j \in \text{valid}} \mathbf{1}\{z_j \neq 0\}, \qquad r_{\text{active}} = K/M, \quad r_{\text{sample}} = K/N, \quad r_{\text{second}} = F_s K/N. \tag{22}$$

The implementation counts exact nonzero float32 values, not values above a post hoc tolerance. For two seconds, $\rho = .01$ gives $K \leq 320$ and $\rho = .10$ gives $K \leq 3{,}200$. With full utterances and final-step padding, requested fractions and K/N differ slightly. In this development probe, mean values per second are approximately 160.28 at 1% and 1,602.06 at 10%.

These are scalar counts, not a bitstream length or a demonstrated compression ratio. A decodable sparse representation needs values, coordinate IDs, length/rate metadata, and the shared decoder. A simple fixed-width accounting proxy would be

$$L_{\text{proxy}} = 32K + K\lceil \log_2 M \rceil + L_{\text{metadata}} \quad \text{bits}, \tag{23}$$

before entropy coding and with no model amortization cost. A combinatorial support code would instead involve roughly $\log_2 \binom{M}{K}$ bits plus framing. Neither code is implemented. A raw float32 decoder contains $32(501{,}057) = 16{,}033{,}824$ parameter bits, reusable across samples; encoder plus decoder contain $32(1{,}002{,}369)$ bits. Serialization overhead and any model quantization are separate. Lower scalar support is useful evidence, but not sufficient to conclude lower total description length.

6.2 Waveform reconstruction

For valid samples, let $e = \widehat{x} - x$, $E = \|x\|_2^2$, and $E_e = \|e\|_2^2$. Evaluation records

$$\text{MSE} = E_e/N, \qquad \text{MAE} = \|e\|_1/N, \tag{24}$$

$$\text{NMSE} = E_e/E, \qquad \text{NRMSE} = \sqrt{E_e/E}, \tag{25}$$

$$\text{SNR} = 10\log_{10}(E/E_e), \qquad \text{logNMSE} = -\text{SNR}. \tag{26}$$

Normalized ratios are undefined for effectively silent references; the code uses mean power below 10^{-10} as its silence threshold and reports null normalized scores, while retaining absolute MSE, MAE, and error RMS. Numerical energy denominators otherwise use a floor of 10^{-20} where needed.

For SI-SDR [8], first mean-center $x_c = x - \overline{x}$ and $y_c = \widehat{x} - \overline{\widehat{x}}$, then use

$$\alpha = \frac{x_c^\top y_c}{\|x_c\|_2^2}, \quad \text{SI-SDR} = 10\log_{10}\frac{\|\alpha x_c\|_2^2}{\|y_c - \alpha x_c\|_2^2}. \tag{27}$$

This exposes distortion apart from a fitted gain, but cannot replace ordinary NMSE/SNR when accurate amplitude matters. No time alignment, gain fitting, or phase correction is applied to the ordinary waveform metrics.

6.3 Spectral, transient, and high-band diagnostics

Let S_q be a centered Hann-window STFT with FFT/window length q and hop $q/4$. PyTorch's centered STFT reflection padding is used after zero-padding very short signals to at least q. Complex STFT NMSE is

$$\frac{\|S_{512}(x) - S_{512}(\widehat{x})\|_F^2}{\|S_{512}(x)\|_F^2}. \tag{28}$$

Unlike magnitude-only scores, this is sensitive to complex phase differences. Writing $A_q = |S_q(x)|$ and $B_q = |S_q(\widehat{x})|$, the reported multi-resolution STFT diagnostic is

$$\frac{1}{3}\sum_{q\in\{256,512,1024\}}\left[\frac{\|A_q - B_q\|_F}{\max(\|A_q\|_F, 10^{-5})} + \text{mean}\left|\ln(A_q + 10^{-5}) - \ln(B_q + 10^{-5})\right|\right]. \tag{29}$$

This diagnostic is not used as a training loss in these runs. Log spectral distance is the time average of the frequency-bin RMS difference of $20\log_{10}(A_{512} + 10^{-5})$ and $20\log_{10}(B_{512} + 10^{-5})$.

Transient NRMSE restricts waveform energy/error to indices whose $|x[n] - x[n-1]|$ reaches the recording's 90th percentile (the first difference is padded with zero). High-band NRMSE is the square root of the real-FFT error/reference energy ratio over bins at or above $F_s/4 = 4$ kHz. Ties can make the transient set exceed exactly 10%; little reference energy can make either ratio unstable. These are explicitly named diagnostic conventions, not separate perceptual quality standards.

6.4 Speech quality and intelligibility

Two optional speech metrics were successfully computed for all 200 examples at each of the eight evaluated budgets:

- **Wideband PESQ:** `pesq==0.0.4`, called with `pesq(16000, x, y, "wb")`. The wideband extension is specified in ITU-T P.862.2 [9]. It is a speech-quality predictor, not a waveform identity criterion or an observed listening-test MOS.
- **STOI:** `pystoi==0.4.1`, `extended=False`; the underlying intelligibility measure is described by Taal et al. [10]. Higher scores indicate better predicted intelligibility under its assumptions. It does not demonstrate exact preservation of the waveform or all audio content.

References shorter than 0.25 s or effectively silent are rejected by the metric wrapper. Missing packages, invalid outputs, and warnings are recorded with explicit status and null values. **ViSQOL is not integrated**; its placeholder is null with status "not integrated; no score computed." These results do not support claiming that every possible quality metric was measured. PESQ and STOI should not be interpreted as general music or environmental-sound scores.

6.5 Aggregation and smallest tested support

For each budget, summaries provide utterance mean, median, 5th/95th percentiles, speaker-macro means, and 95% percentile bootstrap intervals from 1,000 resamples of speaker means. The bootstrap uses seed 0. These intervals describe the macro *mean*, not the median; they do not reflect training-seed variance. With five utterances for every speaker in this probe, utterance means and speaker-macro means coincide.

For each recording and target $\varepsilon \in \{.1, .01, .001\}$, evaluation selects the smallest *actual* support among the eight tested codes whose waveform NMSE is at most ε. These thresholds correspond to SNR at least 10, 20, and 30 dB. All candidates are checked rather than assuming error is monotone with support. If none succeeds, the target is marked unmet. The decoder is not inverted or optimized separately for each support. The returned value is a smallest *tested* support, not an exact minimum even for the fixed decoder.

7 Measured development results

7.1 Completion and checkpoint objective

All runs finished on the NVIDIA RTX 2000 Ada Generation GPU using PyTorch 2.8.0+cu128, Python 3.11.0, and float32. The model states selected as best are all at update 20,000:

Model	Best common validation score $\mathcal{V}$	Peak allocated tensor memory (MiB)
S-NL	0.183015	195.3
D-NL	0.415646	195.3
S-AF	0.103625	130.3

The score is the fixed-training-power objective in Eq. (21), not mean per-utterance NMSE. Reported peak allocation comes from `torch.cuda.max_memory_allocated`; it excludes allocator reservation, context, other processes, and driver overhead. Jobs overlapped in time on one GPU, so their training wall times and recorded inference durations are not isolated architecture-speed benchmarks. High GPU utilization with low VRAM occupancy is possible because utilization measures activity, while memory occupancy measures capacity in use.

7.2 Support–distortion curves

Table 3 uses median utterance SNR and mean utterance NMSE, STOI, and wideband PESQ. The complete 24-row numeric export is `sparse_autoencoder_results.csv` alongside this source; it also includes additional metric summaries and speaker-bootstrap intervals. Results are extracted from saved evaluation files, not inferred from training loss.

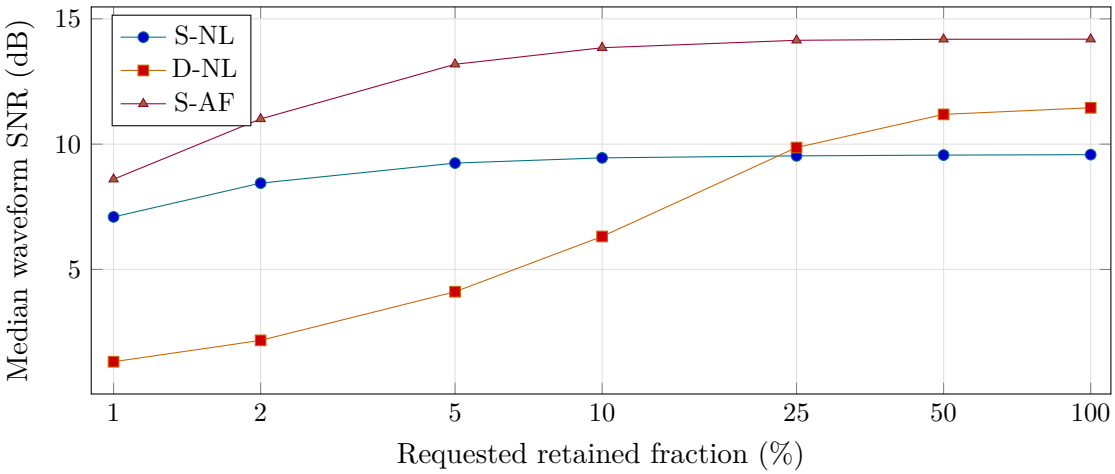


Figure 2: Development reconstruction under common evaluation budgets. The horizontal axis is a requested fraction of valid latent capacity, not a measured bitrate. Lines join evaluated points and do not certify intermediate performance. The zero-code point is omitted from the logarithmic axis.

7.3 Tolerance attainment

Target waveform NMSE	S-NL successes	D-NL successes	S-AF successes
$\leq .1$ (SNR ≥ 10 dB)	90/200	139/200	161/200
$\leq .01$ (SNR ≥ 20 dB)	0/200	2/200	18/200
$\leq .001$ (SNR ≥ 30 dB)	0/200	0/200	0/200

A success means *some* point in the entire eight-budget sweep reaches the target; it does not imply success at the smallest budget. For the 10 dB target, median smallest tested support among successful recordings is 3,566.5 for S-NL, 39,296 for D-NL, and 2,438 for S-AF. These conditional

Model	Retained	Values/s	NMSE	SNR (dB)	STOI	PESQ-WB
S-NL	0%	0.0	1.0001	-0.00	0.440	1.057
S-NL	1%	160.3	0.2180	7.10	0.702	1.250
S-NL	2%	320.5	0.1722	8.44	0.747	1.368
S-NL	5%	801.1	0.1479	9.24	0.781	1.513
S-NL	10%	1,602.1	0.1412	9.45	0.795	1.583
S-NL	25%	4,005.0	0.1379	9.53	0.804	1.638
S-NL	50%	8,010.0	0.1372	9.56	0.808	1.666
S-NL	100%	16,020.0	0.1371	9.58	0.809	1.676
D-NL	0%	0.0	1.0352	-0.04	0.421	1.038
D-NL	1%	160.3	0.7470	1.32	0.468	1.034
D-NL	2%	320.5	0.6141	2.17	0.510	1.040
D-NL	5%	801.1	0.4055	4.11	0.597	1.066
D-NL	10%	1,602.1	0.2551	6.32	0.683	1.121
D-NL	25%	4,005.0	0.1282	9.86	0.799	1.365
D-NL	50%	8,010.0	0.0980	11.19	0.852	1.795
D-NL	100%	16,020.0	0.0929	11.45	0.868	1.931
S-AF	0%	0.0	1.0004	-0.00	0.427	1.076
S-AF	1%	160.3	0.1581	8.60	0.731	1.290
S-AF	2%	320.5	0.1052	11.01	0.798	1.500
S-AF	5%	801.1	0.0771	13.19	0.854	1.832
S-AF	10%	1,602.1	0.0707	13.85	0.876	2.056
S-AF	25%	4,005.0	0.0681	14.14	0.892	2.308
S-AF	50%	8,010.0	0.0677	14.19	0.898	2.450
S-AF	100%	16,020.0	0.0676	14.19	0.900	2.510

Table 3: Common development probe: 200 full utterances, 40 speakers. Values/s, NMSE, STOI, and PESQ-WB are utterance means; SNR is the utterance median. Retained is the requested fraction of valid latent capacity. All scores use best checkpoints selected on this same probe.

medians use different successful subsets and varying recording lengths. They cannot by themselves establish a paired compression advantage. The success counts and full support–distortion curves must accompany them.

7.4 What the observations currently support

1. **Training through sparsification matters at low budgets.** At 1%, S-NL's median SNR is 7.10 dB versus 1.32 dB for D-NL; at 10%, it is 9.45 versus 6.32 dB. These are development observations from one seed, not general statistical conclusions.
2. **Dense quality and sparse robustness trade off.** At full support D-NL reaches 11.45 dB median SNR, above S-NL's 9.58 dB. The sparse model's early plateau shows that increasing the allowed support alone does not ensure near-accurate reconstruction.
3. **The affine control is a serious baseline.** S-AF reaches 13.85 dB median SNR at 10% and 14.19 dB at full support, with better mean STOI/PESQ than both nonlinear models at these points. Greater network nonlinearity has not helped in this particular design and optimization budget. This does not prove affine representations are universally superior.
4. **The stated research goal remains open.** None of the three models reaches 30 dB SNR for any development utterance in the tested sweep. Neither low support nor favorable intelligibility demonstrates that all important details of the waveform have been preserved.

8 Reproduction, files, and code correspondence

8.1 Minimal workflow

Use Python 3.11 or newer and a PyTorch build suitable for the machine. From the repository root, a fresh installation can use:

```
python -m venv .venv
# Activate the environment for your shell, then:
python -m pip install -e ".[dev]"
# Optional, needed to reproduce PESQ/STOI columns:
python -m pip install -e ".[speech-metrics]"
```

Native compilation may be required for the optional PESQ package. The version snapshot in `requirements-lock.txt` records the workstation environment; its CUDA-wheel availability is platform-dependent.

Set `data.root` in `configs/data_librispeech_cached.yaml` to the local corpus/cache root. Official payloads are available at https://www.openslr.org/12. Both standard `LibriSpeech/<partition>` layout and Kaggle's `<partition>/LibriSpeech/<partition>` layout are supported. Do not substitute a single-speaker corpus under the same configuration. Prepare once:

```
python run.py prepare --config configs/librispeech_sparse_autoencoder_10h.yaml
python run.py audit --config configs/librispeech_sparse_autoencoder_10h.yaml
```

Then run each experiment, reusing manifests:

```
python run.py experiment --config configs/librispeech_sparse_autoencoder_10h.yaml --skip-prepare
python run.py experiment --config configs/librispeech_sparse_autoencoder_dense_10h.yaml --skip-
    prepare
python run.py experiment --config configs/librispeech_sparse_autoencoder_affine_10h.yaml --skip-
    prepare
```

These commands prepare no additional speech download, train, evaluate the development probe,

analyze a bounded subset, and write a report in the printed run directory. Running sequentially is sufficient; concurrency is not part of the scientific model definition.

For a saved run, an explicit development sweep is

```
python run.py evaluate --run runs/YOUR_RUN --split validation --limit 200 --keep-fractions
    0,0.01,0.02,0.05,0.1,0.25,0.5,1
```

After freezing the study protocol, a *future* final test would use `-split clean` without a development limit. That final-test command has not been run for this report. The evaluator loads `checkpoints/best.pt` by default. Checkpoints contain model/optimizer states, configuration, training power, update count, and stored RNG state. Resuming restores model/optimizer RNG but restarts the crop/sampler position; an interrupted/resumed trajectory is not promised to match an uninterrupted one bit for bit. GPU numerical kernels can also limit exact cross-machine reproduction.

8.2 Implementation map

Source	Responsibility
`lrac/models/autoencoder.py`	Convolutional stages, residual blocks, decoder ordering
`lrac/models/sparse_autoencoder.py`	Length-safe encoding, global top-k, identity activation control
`lrac/train.py`	Sampling, objective, AdamW, validation, checkpoint selection
`lrac/losses.py`	Masked MSE and spectral diagnostics
`lrac/metrics.py`	Waveform/STFT/speech metrics, speaker bootstrap
`lrac/evaluate.py`	Budget sweep, support counts, tested-support search
`lrac/data/datasets.py`	Speaker coverage, weighted sampling, crops, collation
`lrac/data/manifests.py`	Deterministic subset construction and corpus indexing
`lrac/data/audit.py`	Roles, speakers, source intervals, duplicate-hash audit
`tests/test_sparse_autoencoder.py`	Shapes, padding, code-only decode, gradients, affine control

The existing suite also exercises audits, metric validity, dataset sampling, checkpoint reuse, and the end-to-end workflow. All 61 tests passed during preparation of this report, including the home-path portability check added for repository publication. Documentation and exported results do not replace the executable implementation.

The repository includes the LaTeX source, BibTeX references, compiled PDF, a numeric results export, and a compact run/provenance record. Downloaded recordings, generated path-bearing manifests, environments, checkpoints, and bulky run products are excluded by `.gitignore`. Retraining regenerates local run artifacts; published scalar results alone do not provide downloadable pretrained models.

To rebuild this document from `documentation/`, with a LaTeX distribution including BibTeX, TikZ, and PGFPlots:

```
latexmk -pdf -interaction=nonstopmode -halt-on-error sparse_autoencoder_models.tex
```

9 Limits and next research decisions

9.1 What the baseline does not establish

- **Global sparsestness:** encoder inference and magnitude top-k do not solve Eq. (1) over arbitrary decoders or even all codes for the trained decoder. General fixed-dictionary sparse approximation is already computationally difficult [11]; that result does not prove an impossibility theorem for every restricted neural formulation.
- **Exact reconstruction or invertibility:** no Jacobian, frame-bound, or perfect-reconstruction constraint is enforced. Full-budget residual error is substantial.
- **Per-sample optimal adaptation:** the learned bank of convolutional parameters is shared. Only latent values and positions adapt. There is no search over input-dependent transforms, learned stopping rule, per-example latent optimization, or certified lower bound.
- **Bitrate and arbitrary precision:** values remain float32 and no entropy-coded bitstream is produced. A one-number code with unbounded numerical precision would be an accounting loophole, not evidence of physical compression.
- **General audio generalization:** the main data are clean English read speech, one training seed, and a limited development probe. Speaker separation does not establish transfer to all signals.
- **Semantic or independent features:** a coordinate is an operational reconstruction variable. Neither semantic interpretation, orthogonality, nor independence is required or demonstrated.

9.2 Questions made sharper by the three runs

The first priority is to explain the full-budget reconstruction floor before treating sparse quality as a fundamental limit. Useful controlled tests would vary downsampling/receptive field, initialization, training duration, and waveform versus spectral objectives while retaining common data and checkpoint criteria. The current affine advantage motivates a simple perfect-reconstruction or invertible analysis–synthesis reference rather than assuming that additional nonlinear depth is the answer.

Next, compare hard magnitude ranking with reconstruction-aware support refinement under a *fixed* decoder and fixed budget. For S-AF, optimize selected coefficients and test alternative supports; for S-NL, separate amplitude optimization from support changes. Any gains would quantify the inference gap between this encoder and better codes within the same decoder, without conflating that gap with model capacity. Analyze paired recordings at matched distortion and report failures as well as successes.

Finally, repeat with multiple seeds, use the remaining development speakers' utterances carefully, and freeze the selection procedure before final test evaluation. Synthetic signals remain useful because known generative structure can expose failures, but success on selected synthetic families would not establish universality. Quantization, support coding, and shared-model accounting should be added when comparing actual description lengths. These questions keep the experiment connected to the dream system while avoiding claims the current evidence cannot support.

References

[1] J. Rissanen. Modeling by shortest data description. *Automatica*, 14(5):465–471, 1978. doi: 10.1016/0005-1098(78)90005-5.

[2] Alireza Makhzani and Brendan Frey. k-Sparse Autoencoders, 2014. URL https://arxiv.org/abs/1312.5663v2. arXiv:1312.5663v2, revised 22 March 2014.

[3] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016. doi: 10.1109/CVPR.2016.90. URL https://arxiv.org/abs/1512.03385.

[4] Saiprasad Ravishankar and Yoram Bresler. Learning sparsifying transforms. *IEEE Transactions on Signal Processing*, 61(5):1072–1086, 2013. doi: 10.1109/TSP.2012.2226449. URL https://transformlearning.csl.illinois.edu/assets/Sai/JournalPapers/SaiTSP2013Square.pdf.

[5] Luke Pfister and Yoram Bresler. Learning filter bank sparsifying transforms. *IEEE Transactions on Signal Processing*, 67(2):504–519, 2019. doi: 10.1109/TSP.2018.2883021. URL https://arxiv.org/abs/1803.01980.

[6] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019. URL https://arxiv.org/abs/1711.05101v3.

[7] Vassil Panayotov, Guoguo Chen, Daniel Povey, and Sanjeev Khudanpur. LibriSpeech: An ASR corpus based on public domain audio books. In *IEEE International Conference on Acoustics, Speech and Signal Processing*, pages 5206–5210, 2015. doi: 10.1109/ICASSP.2015.7178964. URL https://www.danielpovey.com/files/2015_icassp_librispeech.pdf.

[8] Jonathan Le Roux, Scott Wisdom, Hakan Erdogan, and John R. Hershey. SDR – half-baked or well done? In *IEEE International Conference on Acoustics, Speech and Signal Processing*, pages 626–630, 2019. doi: 10.1109/ICASSP.2019.8683855. URL https://arxiv.org/abs/1811.02508.

[9] International Telecommunication Union. Wideband extension to recommendation P.862 for the assessment of wideband telephone networks and speech codecs. Technical Report Recommendation P.862.2 (11/2007), ITU-T, 2007. URL https://www.itu.int/rec/dologin_pub.asp?id=T-REC-P.862.2-200711-W!!PDF-E&lang=e&type=items.

[10] Cees H. Taal, Richard C. Hendriks, Richard Heusdens, and Jesper Jensen. An algorithm for intelligibility prediction of time–frequency weighted noisy speech. *IEEE Transactions on Audio, Speech, and Language Processing*, 19(7):2125–2136, 2011. doi: 10.1109/TASL.2011.2114881. URL https://ceestaal.nl/Taal(2011).pdf.

[11] B. K. Natarajan. Sparse approximate solutions to linear systems. *SIAM Journal on Computing*, 24(2):227–234, 1995. doi: 10.1137/S0097539792240406.